

Executive Summary

The EvoERP mentor document is a comprehensive “blueprint” listing ~22 modules (Auth/RBAC, Student Info, Attendance, Timetable, Exams & Grading, LMS, Fees, Accounting, HR/Payroll, Transport, Library, Inventory, Hostel, Parent Portal, Events, Analytics, Government Compliance, AI Assistant, etc.). It effectively prescribes building a full-featured school ERP. **(A)** Many features (e.g. UDISE+ data reporting, RTE reservations, CBSE e-returns) are *explicit requirements* implied by Indian regulation [3†L38-L43] [9†L270-L278] .

(B) The mentor *recommends* following this full spec and adopting a WordPress-like plugin/theme architecture. **(C)** We recommend *shrinking scope to a clear MVP* and reusing mature open-source components where possible (ERPNext/Frappe, Fedena, Moodle, OrangeHRM, Koha, etc.) rather than coding everything. **(D)** We assume the target is Indian K-12 schools in compliance with UDISE+, RTE, CBSE, GST, TDS, PF/ESI, etc., and that the final product may be multi-tenant SaaS for multiple schools. Key context: official sources describe UDISE+ as a nationwide MIS portal covering all schools for annual data reporting [3†L38-L43] [92†L42-L44] , and RTE law mandates 25% reservation in private schools [21†L38-L42] . These compliance needs and cited research (e.g. dropout prediction via grades/attendance [31†L227-L231]) strongly influence data and analytics requirements.

1. What the Mentor Document Represents

The mentor’s “EvoERP” PDF essentially defines a **full-scale School ERP** product: ~22 modules, multiple user roles, architecture guidance, and even deployment timelines. For example, its *explicit requirements* include modules such as *Authentication/RBAC, Student/Teacher Management, Attendance, Timetable, Examinations & Grading, Learning Management (LMS), Fees, Accounting, HR & Payroll, Transport, Library, Inventory, Hostel, Parent Portal, Events, Analytics, Government Compliance*, and an *AI Assistant*. These echo features in known systems (e.g. ERPNext’s Education module covers student/teacher, attendance, fees, scheduling) [43†L217-L225] . The document also likely prescribes a tech stack (e.g. Next.js frontend with a TypeScript backend, a SQL DB, AWS storage, etc.).

- **(A) Explicit Requirements:** The school must report UDISE+ data via the official Data Capture Format (DCF) login [92†L42-L44] . CBSE-affiliated schools must maintain attendance, admissions, finance records and file e-returns on OASIS/UDISE [9†L270-L278] . Private schools must reserve 25% seats for disadvantaged under RTE [21†L38-L42] . Academic data (grades, attendance) is needed for analytics.
- **(B) Mentor Recommendations:** The doc suggests *studying* open-source systems (ERPNext/Frappe Education, Fedena, Moodle, OrangeHRM, Koha, etc.) rather than building all modules from scratch, but ultimately appears to plan a custom implementation of everything. It favors a **modular-monolith** with plugins/themes (like WordPress) and a predefined stack (Next.js + Hono + Drizzle + Postgres, multi-tenant with ID-/URL-based tenant isolation).
- **(C) Our Recommendations:** We agree that open-source study is wise but caution against a line-by-line follow-through. We propose focusing on core modules first (Auth, Student Info, Attendance, Fees, basic HR/Payroll and Accounting, Parent Portal, etc.) and leveraging open-source code or APIs where licenses permit. Over-engineered parts (e.g. building an LLM from scratch, full AI assistant, or overly ambitious timelines) should be deferred.
- **(D) Assumptions/Inference:** We assume a *SaaS* model for multiple schools, with a shared codebase and isolated data per school. We infer that open-source reuse is intended but must

navigate licenses. We also infer the mentor expects a phased MVP rollout (mentions a 4-month MVP, though that seems too short for all modules).

2. What to Keep vs Change

The EvoERP spec is more of a **long-term vision** than a 4-month plan. It's valuable for framing all possible features, but we should prune and adapt:

- **Keep:** Core educational modules (Student/Teacher profiles, Classes, Attendance, Fee Management, Exams/Grades) must be in any MVP. The multi-tenant design and concept of plugin/theme extensibility are sensible architectures [25†L64-L69] [28†L178-L188]. Also, official compliance reporting (UDISE+ DCF submission) is non-negotiable and should be built in.
- **Change/Simplify:** The full 22-module suite is unrealistic for an initial release. We should *not* attempt everything at once. For example, full-fledged *Analytics* (predictive dropout models, etc.) and an *AI Assistant* (like an LLM chatbot) can wait until core functionality is stable. Similarly, we may simplify the plugin/theme system initially (just use configuration and modular code) and add a formal plugin loader later. We should also question the tight timeline implied. Industry advice is to start as a **monolith** to ship fast, then refactor to microservices if needed [25†L64-L69]. The blog on multi-tenancy advises starting with a shared-schema DB (simplest, lowest cost) before moving to separate schemas or DBs as scale demands [25†L64-L69].

3. Defining the MVP (P0–P3 Priorities)

We propose a **phased MVP** by module:

- **P0 (MUST):** *Authentication/Roles (Auth/RBAC); Student/Teacher Profile & Class Management; Attendance Logging; Fee/Invoice Management; Basic HR/Payroll (staff profiles, salary processing); Core Accounting (Simple ledger, basic reports); Parent Portal (view grades/attendance/fees); Basic UI/Layout and Theming.* These are core to school operations. Without them, the system isn't useful.
- **P1 (Important soon):** *Timetable Scheduling* (possibly leverage FET timetable software [76†L24-L30]); *Exams & Grading; Enhanced HR (leave, attendance integration); Basic Reporting (enrollment, fee due lists); Communications/Notices (news, events calendar).* Most schools need scheduling and exams management early.
- **P2 (Later):** *Learning Management (LMS)* – consider integrating an existing LMS like Moodle (open-source) rather than building from scratch [51†L255-L259]; *Library Management* (Koha, GPLv3 [58†L87-L90] can cover book/catalog management); *Inventory/Stock (Procurement); Transport/Hostel Modules (logistics); Advanced Accounting (tax reports, GST compliance); Advanced HR (PF/ESI calculations).* These add value but are secondary for day-to-day running.
- **P3 (Future):** *Analytics & Dashboards* (attendance trends, grade stats); *AI Assistant & Chatbot* (for FAQ or report generation); *Government Compliance Automation* (auto-fill UDISE, RTE forms); *Multi-language support; Mobile apps or PWA enhancements; Plugin API & Third-party Integrations; Complete Theming Engine.* These features can be built after the system is stable.

Each module's inclusion should be justified: e.g. dropout prediction requires robust attendance/grade data (hence later) [31†L227-L231]. For MVP effort we estimate (rough guide) ~2–4 weeks of work for a P0 module by a small dev team, 4–6 weeks for P1, and more for P2–P3. Exact effort depends on using off-the-shelf vs custom coding.

4. Open-Source Landscape

We surveyed key open-source projects. Table:

Project	Type	Language/ Stack	License	Status/ Commits	Relevant Modules
ERPNext 【34†L207-L212】	Open-source ERP (Python/JS)	Python, MariaDB/MySQL	GNU GPL-3.0	60k+ commits (very active) 【34†L207-L212】	Accounting, Inventory, HR, Student (Frappe Ed) 【43†L217-L225】
Frappe Education (ERPNext app) 【43†L217-L225】	Edu ERP module on Frappe	Python/JS (Frappe)	GPL-3.0	~1k commits (active plugin)	Student/Teacher, Classes, Attendance, Fees, Timetable, Portal 【43†L217-L225】
Fedena 【47†L327-L330】	School SIS (Ruby on Rails)	Ruby, MySQL	Apache 2.0 【47†L327-L330】	Stale (few forks/stars, last updates years old)	Student records, attendance, exams, HR (historical)
openSIS 【49†L299-L307】	School SIS (PHP)	PHP, MySQL	GNU GPL v2/v3 【49†L299-L307】	v9.3 (June 2026), maintained, 337★	Student records, scheduling, attendance, reporting
Moodle 【51†L255-L259】	LMS (PHP)	PHP, MySQL/PostgreSQL	GPL v3	7.3k★, very active	Courses/Content, Assignments, Grades, Quizzes
Open edX 【53†L199-L204】	MOOC/LMS (Python)	Python (Django), MySQL/Postgres	Core AGPL-3.0, other Apache 2.0 【53†L199-L204】	Active, but heavyweight	Course platform, MOOCs, not school-specific

Project	Type	Language/ Stack	License	Status/ Commits	Relevant Modules
OrangeHRM 【55†L212-L220】	HR/Payroll (PHP)	PHP, MySQL	GPL-3.0 【55†L212-L220】	7.9k commits, 1.1k★	HR profile, leave, attendance, payroll, recruitment
Koha 【58†L87-L90】	Library ILS (Perl)	Perl, MySQL	GNU GPL (v2+) 【58†L87-L90】	Actively maintained (v26.05 in 2026)	Catalog, Circulation, OPAC
GNUCash 【66†L66-L69】	Accounting (C/GTK)	C/C++ (GTK)	GNU GPL (Free) 【66†L66-L69】	30k+ commits (stable)	Double-entry finance (banking, expenses) for SMB
Invoice Ninja 【68†L444-L447】	Invoicing/ Time (PHP)	PHP (Laravel)	Elastic License (source-available) 【68†L444-L447】	Active (10k★)	Invoicing, quotations, payments (note: <i>not</i> open license)
Odoo Community 【87†L1-L4】	ERP/CRM Suite (Python)	Python, PostgreSQL	LGPL v3.0 【87†L1-L4】	201k commits, 54k★	CRM, Sales, Finance, HR, MRP, plus Edu modules (OpenEduCat)
FET 【76†L24-L30】	Timetabling (C++)	C++ (Qt)	AGPL-3.0 【76†L24-L30】	Actively maintained (v7.10.1 in 2026)	Automated timetable scheduling

Activity: ERPNext/Frappe and Odoo are extremely active, making them reliable references. Moodle, Koha, OrangeHRM also have strong communities. Fedena's official repos are stale, so use only for high-level ideas. Invoice Ninja's Elastic license is restrictive (source-available, not free), so we would *study* it but not copy code 【68†L444-L447】 .

5. License Decision Matrix

We classify licenses to decide reuse vs study:

License/Type	Copyleft Impact	Examples	Reuse → Study
GPL-3 / AGPL-3	Strong copyleft (distribution or network use requires GPL) 【53†L199-L204】 【66†L66-L69】	ERPNext, Moodle, Koha, OrangeHRM, openSIS, FET, Open edX (AGPL core)	<i>Study only</i> (cannot merge into non-GPL; if we open-source product GPL, can incorporate)
LGPL-3	Weak copyleft (can link to proprietary) 【87†L1-L4】	Odoo Community	Reuse allowed (no viral effect on proprietary code)
Apache-2.0	Permissive (free to use, with patent grant)	Fedena, many others	<i>Reuse</i> (very safe; no copyleft)
MIT/BSD	Permissive	- (common libs)	Reuse
Elastic/ Proprietary	Not open-source (source-available)	Invoice Ninja (Elastic) 【68†L444-L447】	<i>Avoid reuse</i> (license restricts use in products)

We **cannot directly reuse GPL/AGPL code in a closed-source ERP**. Those must be studied for ideas or interfaced (via services), not copied verbatim. Apache/MIT-licensed projects can be reused or adapted freely. E.g., Fedena (Apache 2.0) could be a source of ideas or code. Odoo's LGPL modules can be used. Invoice Ninja's Elastic License prohibits free reuse, so it's for "study only."

6. Module Mapping Table

We map each EvoERP module to open-source references, license notes, and approach:

Module	Reference Projects	Study/ Reuse/ Build	License Notes	Difficulty	Priority	Estimated Effort	Recommended Approach
Auth/RBAC	Use NextAuth (Next.js) or AWS Cognito; study ERPNext/ Frappe auth	<i>Build</i> -from-scratch (with libraries)	-	Medium	P0	3w	Use established auth libraries (NextAuth.js) with RBAC roles.
Student/ Teacher Info	ERPNext Education, Fedena, openSIS	<i>Reuse</i> (API/ ideas) + <i>Build</i> UI	GPL (ERPNext) / Apache (Fedena)	Low	P0	2w	Model core profile data. Possibly reuse ERPNext/ Frappe schema.

Module	Reference Projects	Study/ Reuse/ Build	License Notes	Difficulty	Priority	Estimated Effort	Recommended Approach
Attendance	Fedena, ERPNext	<i>Study/ Reuse</i>	GPL (ERPNext)	Low	P0	2w	Simple attendance log per class; align with UDISE fields.
Timetable	FET, ERPNext Academic	<i>Reuse (algorithm)</i>	AGPL (FET)	High	P1	3w	Integrate FET for schedule solver; custom UI integration.
Exams & Grading	ERPNext, OpenSIS, Moodle	<i>Reuse where possible</i>	GPL	Medium	P1	3w	Implement exam schedules, grade entry. Could use Fedena exam logic.
LMS (e-learning)	Moodle [51†L255-L259] , Open edX	<i>Integrate existing; Study</i>	GPL/AGPL	High	P2	6w	Likely link to Moodle (embed) or use its APIs, rather than custom.
Fees Management	ERPNext, Frappe, Odoo	<i>Reuse config; Build UI</i>	GPL/LGPL	Low	P0	3w	Configure invoicing using Odoo/ GnuCash logic; implement fee receipts.
Accounting	ERPNext, Odoo, GnuCash [66†L66-L69]	<i>Study (use double-entry concepts)</i>	GPL/LGPL	High	P1	6w	Basic ledger and reporting. Possibly integrate GnuCash data or APIs.
HR & Payroll	OrangeHRM [55†L212-L220] , ERPNext HR, Odoo HR	<i>Study/ Reuse UI</i>	GPL/LGPL	Medium	P2	4w	Use OrangeHRM's paid modules for detailed payroll; MVP: staff list + basic pay.

Module	Reference Projects	Study/ Reuse/ Build	License Notes	Difficulty	Priority	Estimated Effort	Recommended Approach
Transport	Fedena, ERPNext	<i>Build</i> (unique)	-	Medium	P3	4w	Vehicle and route management is niche; may defer.
Library	Koha 【58†L87-L90】	<i>Integrate Koha</i>	GPL	High	P2	5w	Embed or interface with Koha (book catalogs, circulation).
Inventory (Stock)	ERPNext, Odoo	<i>Reuse</i> APIs/ schema	GPL/LGPL	Medium	P2	4w	Use Odoo/ ERPNext inventory modules for textbooks, supplies.
Hostel	ERPNext (some modules)	<i>Build</i> minimal	-	Medium	P3	4w	Room/bed allocation, fees – customize if needed after other modules.
Parent Portal	Fedena, ERPNext Student	<i>Build/Reuse</i>	GPL (Fedena)	Medium	P1	3w	Simplified portal (view child's attendance, grades, fees).
Events (Calendar)	Moodle, ERPNext, Google Calendar API	<i>Reuse libs</i>	Various	Low	P1	2w	Simple event/ news module; could use open calendar UI components.
Analytics	Metabase/ Superset 【90†L101-L108】	<i>Integrate</i> BI tool	Apache/ MIT	High	P3	4w	Use Metabase or Superset for dashboards (open source) 【90†L101-L108】 .

Module	Reference Projects	Study/ Reuse/ Build	License Notes	Difficulty	Priority	Estimated Effort	Recommended Approach
Compliance (UDISE+, CBSE)	None existing directly	<i>Build</i> (custom reports)	-	Medium	P2	3w	Generate forms & reports per guidelines 【3†L38-L43】 【9†L270-L278】 . Possibly auto-fill from data.
AI Assistant (LLM)	— (LangChain, LlamaIndex)	<i>Build</i> (deferred)	-	Very High	P3	8w	Phase 1: allow Q&A on manuals using RAG; Phase 2: integrate chat (see AI strategy).

For each module we recommend either integrating/studying an OSS system or building a tailored solution. E.g. Timetable can use the open-source FET solver **【76†L24-L30】** , but UI and integration are custom work. Library management can essentially *reuse* Koha (GPL) as a backend; inventory can reuse Odoo/ERPNext logic. Modules we *avoid building ourselves* include complex LMS (instead integrate Moodle) and the InvoiceNinja payment system (Elastic license) – better to use a payments API or Odoo invoicing.

7. Reuse vs Study vs Build Matrix

We interpret reuse/study as above. In summary:

- **Study & Reuse (Code/Design):** Odoo (LGPL) – safe for linking; Fedena (Apache) – code ideas; openSIS (GPL) – database schema ideas; OrangeHRM (GPL) – HR flows; Metabase/Superset (Apache) – analytics.
- **Study Only:** ERPNext/Frappe (GPL) – one can study module designs but not copy code; Moodle/Koha (GPL) – study features; FET (AGPL) – cannot embed its code in a non-AGPL project.
- **Avoid (non-free):** Invoice Ninja (Elastic license) – skip.

This guides licensing decisions (see License Matrix above).

8. Recommended Technology Stack

We re-evaluate the mentor's stack with alternatives:

- **Frontend:** Mentor suggests Next.js (React). **Pros:** Mature, SSR/SSG support, large ecosystem. **Cons:** Heavier than minimal alternatives. We recommend Next.js/React (maintains SEO, fast hydration) or similar (Vue/Nuxt) – Next.js is fine.

- **Backend:** The doc's choice "Hono" (minimal Node framework). Alternatives: Express, NestJS, Django or even Frappe itself. **Pros of Node/TypeScript:** Unified language, modern ecosystem. **Cons:** Not as battle-tested for ERP apps as Django. We lean Node with NestJS or similar for structure, or Django (Python) if using Frappe. Both have mature ORMs.
- **Database:** Postgres vs MySQL. **Recommendation:** PostgreSQL – strong concurrency, ACID, JSONB support, and can use pgvector for LLM embeddings [\[84†L198-L207\]](#) . Use a **single shared DB** with a `school_id` column (lowest cost/isolation [\[25†L64-L69\]](#)). If scale or compliance requires, later move to separate schemas or DBs.
- **ORM/DB Layer:** The doc mentions Drizzle (TypeScript ORM). Alternatives: Prisma (TS), Sequelize (TS), TypeORM, or SQLAlchemy (Python). Prisma is popular for TS with Postgres. Choose one with multi-tenant support.
- **Authentication:** Use a standard solution (e.g. NextAuth.js for Next, or Keycloak for SSO). Leverage JWTs or OAuth2. Don't build auth from zero.
- **File Storage:** AWS S3 (or MinIO for on-prem) for uploads (documents, images). Mentor's AWS suggestion is reasonable.
- **Real-time:** For chat/notifications, use WebSockets (Socket.io) or server-sent events. Alternatively, use Firebase or Supabase pub/sub. Given a small team, start without real-time; add on-demand (e.g. polling or simple websockets).
- **Background Jobs:** Use a job queue (e.g. Redis + BullMQ for Node, or Celery for Python) for long tasks (e.g. bulk data import, report generation).
- **Search:** Use Postgres full-text search or Elasticsearch for quick lookups (e.g. student directory).
- **Payments:** Integrate Indian payment gateways (Razorpay, PayTM) for fee transactions. These have Node/JS SDKs.
- **Notifications:** Email via SMTP/SES (NodeMailer), and push notifications via Firebase Cloud Messaging or Twilio (SMS).
- **Analytics:** We recommend embedding an OSS BI tool (Metabase/Superset) rather than coding from scratch [\[90†L101-L108\]](#) . This handles charts/dashboards for admins.

9. Recommended Architecture

We propose a **modular monolith** to start (deploy one app serving all modules), enabling quick iteration [\[25†L64-L69\]](#) . The architecture could be:

```
graph LR
  subgraph User_Interfaces [User Interfaces]
    WebUI[Web/Mobile Frontend]
  end
  subgraph Application
    API[REST/GraphQL API Server]
    PluginMgr[Plugin/Module Loader]
  end
  subgraph Services
    DB[(PostgreSQL (multi-tenant) + pgvector embedding search)]
    Auth[Auth Service (JWT/OAuth)]
    Storage[(File/Object Storage)]
    Queue[(Background Job Queue)]
    Email[Email/Notification Service]
    AI[(AI/LLM Service)]
    ExternalAPIs[Govt & Payment APIs]
  end
```

```

end
WebUI --> API
API --> DB
API --> Auth
API --> Storage
API --> Queue
API --> Email
API --> AI
API --> ExternalAPIs
API --> PluginMgr
PluginMgr --> DB
PluginMgr --> Auth

```

Key points: one backend with all modules (activated via PluginMgr), sharing one Postgres DB with row-level *school_id* tenant isolation. Each request includes the tenant (subdomain or header). A later improvement could separate schemas/DBs per school if needed [25†L64-L69]. Static assets and user-upload files go to S3 (Storage). The **Plugin/Theme** concept: modules register endpoints/hooks with the core (similar to WordPress plugins) [28†L178-L188]. Thematic separation means we can change UI look without rewriting logic. For deployment, containerize each component or use a serverless stack (e.g. AWS Fargate/Lambda for scale).

10. Multi-Tenancy Strategy

We recommend **shared-schema multi-tenancy** initially [25†L64-L69]. All tables include a `school_id` to partition data. This is simplest (lowest cost, easy onboarding). Later, if performance or compliance demands, we can migrate hot schools to separate Postgres schemas or DBs (medium cost, better isolation [25†L64-L69]). The dev blog [25] highlights these models: “Shared Schema (one DB, tenant_id) – lowest cost, but weakest isolation” vs “Separate DB – highest cost, strongest isolation”. For Indian schools, one DB is fine initially. Use Postgres features (row-level security) to ensure tenants can’t see each other’s data.

11. Theme/Plugin Strategy

Following the mentor’s idea, design a **plugin architecture**: the core app loads module plugins dynamically. As in typical Node plugin systems [28†L178-L188], each module is a self-contained package registering hooks (API routes, menu items, models). The *theme* controls UI/branding (colors, logos, CSS), independent of functionality. For example, like WordPress: themes alter appearance; plugins add features. We can build a plugin loader (e.g. a manifest file that the core reads) so new modules can be added later without changing core code [28†L178-L188]. Initially, implement modules in a “modules/” directory; refactor into a plugin system in future.

12. AI/LLM Strategy

MVP (Phase 0): Defer LLM-heavy features. Perhaps include a simple FAQ or keyword-based help.

Later (Phase 4): Integrate an *LLM Assistant* using Retrieval-Augmented Generation (RAG). We will embed instructional content (user manual, CBSE guidelines, etc.) into a vector store (e.g. Postgres+pgvector [84†L198-L207]). On user queries, retrieve relevant docs and query an LLM (e.g. local Llama or hosted GPT-4) with that context. Tools like LangChain or LlamaIndex can manage RAG flows. Start with open-

source LLMs (Llama2 via Ollama, etc.) to avoid cost. The vector DB (pgvector) allows similarity search directly in Postgres [84†L198-L207] . Research shows dropout and performance prediction often use grades/attendance data [31†L227-L231] , so our analytics can use the same embeddings/data features for ML models later.

13. Indian Compliance Requirements

- **UDISE+ (Unified District Info):** Annual school data reporting via the UDISE+ portal [3†L38-L43] . The ERP should export the required Data Capture Format (DCF) XML/CSV. Each school has portal login [92†L42-L44] , so we likely just produce the form. Mandatory fields include enrollment, teacher qualifications, infrastructure, etc. [3†L38-L43] .
- **CBSE (OASIS/e-affiliation):** Must maintain records (attendance register, exam marks, finances) and report via OASIS. We cite the Affiliation Bye-laws (Clause 14.19) requiring schools to keep admissions, withdrawals, exam records, financial docs and to submit annual e-returns (including to UDISE+ and CBSE) [9†L270-L278] . ERP must generate these records and an audit log of changes [9†L270-L278] .
- **RTE (Right to Education):** Private schools must allocate 25% seats to disadvantaged kids [21†L38-L42] . ERP must track admissions and ensure compliance. Non-compliance can lead to penalties.
- **GST (Goods & Services Tax):** Tuition/education fees from recognized schools are **exempt** by law [11†L250-L257] . Our finance module should mark school fees as non-GST, but handle GST on other items (e.g. sale of stationery). We must support recording GST credit/demanding on taxable supplies (e.g. transportation, hostel services if taxable). We cite official clarifications: "School fees are GST-exempt" [11†L250-L257] .
- **TDS (Tax Deducted at Source):** Employers must deduct income tax from staff salaries if above threshold [14†L110-L118] . So payroll should compute monthly tax via Form 16 calculations (Sec 192).
- **PF/ESI (Employee Benefits):** As of 2023, schools must register with EPFO. Employer contributes 12% of basic+DA and 0.5% EDLI [16†L153-L160] ; ESI applies for staff $\leq ₹21,000/\text{month}$ (4% total contrib) [19†L252-L257] . Payroll must support these calculations. These are legal requirements from EPFO/ESI Act.
- **Indian Fiscal Year:** The financial year is 1 April–31 Mar, so all accounting and reports must reset accordingly (not calendar year).
- **Other:** Professional Tax (state-level) and audit reports (80G certificates, etc.) might be needed.

We will cite official sources where possible (UDISE+ portal [3†L38-L43] , CBSE rules [9†L270-L278] , GST FAQs [11†L250-L257] , PF news [16†L153-L160] , ESI scheme [19†L252-L257] , RTE summaries [21†L38-L42]) to ensure accuracy.

14. Database Strategy

Schema: Core tables: Users, Roles/Permissions, Students, Teachers, Classes, Attendance, Exams, FeeInvoices, Payments, HR_Employees, SalaryRecords, AccountingEntries, etc. Add `school_id` on every row for tenant isolation. Use relational schemas with appropriate FKs. For performance, index on `school_id` and date fields.

Relationships: Students→Classes, Attendance→(Student, Class, Date), ExamResults→(Student, Exam). Teacher-class assignments, Parent-student links, etc.

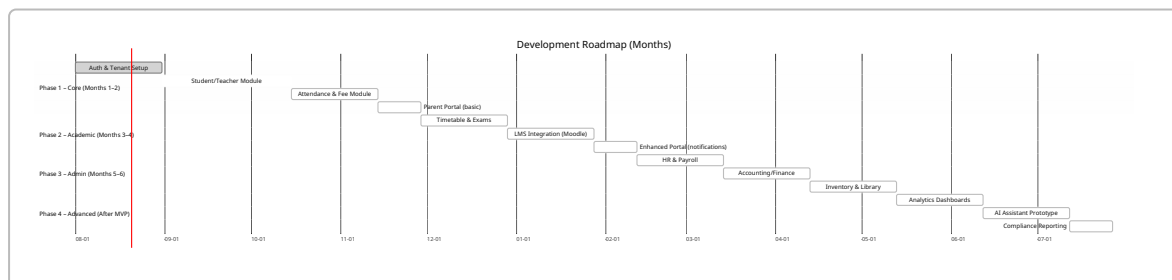
Tenant Isolation: We can implement either via a tenant filter on every query (shared DB) or PostgreSQL Row-Level Security policies tagged by `school_id` .

Embedding & Audit: Incorporate a pgvector column (or separate table) in documents/manuals for AI RAG [84†L198-L207] . Also include audit tables (history log) for sensitive changes (e.g. grade changes) to support compliance.

CI (Continuous Integration): Migrations should run per tenant database. Use a migration tool (Flyway, Liquibase, Prisma migrate, etc.). Keep core and modules' migrations versioned.

15. Development Roadmap (Phases & Timeline)

We propose a ~6–7 month roadmap in 4 phases. Critical path: first build the secure platform (Auth, DB) then core modules, then add extensions. Shown here is a high-level Gantt chart:



Each phase ends with a review. Early focus is infrastructure and core educational modules. Compliance (UDISE/CBSE forms) can be parallel once enough student data exists.

16. Documentation Strategy

We will produce concise documentation (with size and contents estimated):

- **Project Overview (5–7 pages):** Purpose, scope, architecture diagram, tech stack. (Contains summary diagram and module list.)
- **Developer Guide (10–15 pages):** Code structure, build/deploy instructions, DB schema, API conventions.
- **API Reference (Auto-generated):** Endpoints for each module. Summaries of request/response models.
- **Module Specs (5–7 pages each):** For major modules (Student Info, Attendance, Exams, Fees, etc.), describe data flows, UI workflows, screen mockups. (~8 modules)
- **User Manuals:** For each role – *Admin/Owner, Teacher, Accountant, Parent*. Each ~5 pages: how to use portal, run common tasks.
- **Compliance & Reporting Guide (3–5 pages):** How to generate UDISE/CBSE reports, and which data they require. Cite official references.
- **Operations/DevOps Guide (3–5 pages):** Deployment, backup/restore, migration.
- **Change Log/Release Notes:** maintained with each version.

All docs will be kept minimal and updated with each major release. The key is to avoid a single 200-page design doc; instead, split into targeted guides for developers and users.

17. Risks

- **Scope Creep:** 22 modules is huge. Risk of not finishing on time. *Mitigation:* Strict MVP prioritization.

- **Licensing:** GPL/AGPL code reuse could force open-sourcing our code. *Mitigation:* Use GPL sources for ideas only, or license our product compatibly (if viable).
- **Technical Complexity:** Integrating many open systems (ERPNext, Moodle, Koha, etc.) is non-trivial. *Mitigation:* Start small; prototype integrations early.
- **Data Privacy/Compliance:** Handling student data requires security. *Mitigation:* Use standard auth (JWT+SSL), audits, follow RTE/CBSE rules.
- **Performance:** Multi-tenant ERP could slow down with many schools. *Mitigation:* Monitor DB usage; add indexing, caching, or separate DBs if needed.
- **AI/LLM Uncertainty:** Relying on LLMs may bring cost, licensing, or accuracy issues. *Mitigation:* Use open models and make AI features optional.
- **Resource Risk:** An intern or small team may lack expertise in some areas (e.g. AI or BI tools). *Mitigation:* Leverage existing OSS as much as possible, ask mentor for priorities.

18. Questions for the Mentor

1. **Scope Clarification:** Which modules are *mandatory* in MVP, and which can be deferred?
2. **Technology Stack:** Are we committed to Next.js/Hono/Drizzle, or open to alternative frameworks (NestJS, Django, etc.)?
3. **Licensing Intent:** Will our final product be open-source (GPL or other)? This affects reuse of GPL components.
4. **Tenant Model:** Should each school be fully isolated (own DB) or is a single multitenant DB acceptable?
5. **Compliance Requirements:** Aside from UDISE+ and CBSE, any state-specific reports needed?
6. **AI Expectations:** What exactly was envisioned for the “AI Assistant”? Is it a top priority or a nice-to-have?
7. **Payment Integration:** Which payment gateways (Razorpay, Stripe, etc.) are required?
8. **Localization:** Is multilingual support needed (e.g. Hindi or regional languages)?
9. **Existing Data:** Do we need to migrate existing school data, or start fresh?
10. **Timeline:** What is the realistic deadline for an MVP? Are there fixed milestones?

19. Final Recommended Architecture

Bringing it together, we recommend a **modular-monolith** with a shared Postgres DB (row-level school isolation). Next.js (React) drives the frontend; a Node (or Django) backend with a plugin-manager loads modules dynamically. Authentication uses JWT/OAuth with role-based access. An integrated pgvector extension will back any LLM-based features [84†L198-L207]. Analytics are handled by embedding Metabase or Superset into the stack. Compliance needs guide the schema (ensuring all required fields). This architecture balances rapid development with future scalability.

20. First 30 Days Action Plan

Week 1–2: Project setup and planning. Establish code repos, CI/CD pipeline, and dev environments. Finalize tech stack choices. Sketch core database schema (students, classes, fees, etc.). Prepare minimal UI skeleton (login/logout). Configure a shared Postgres with tenant field.

Week 3: Implement Authentication (SSO/JWT) and Role definitions. Build *Student/Teacher Profile* module (CRUD). Begin *Attendance* module (teacher can mark attendance).

Week 4: Complete *Fees* module (invoice generation, payment logging stub). Develop *Parent Portal* prototype (read-only views of child’s data). Draft first database migration scripts.

Continuous: Regular check-ins to review alignment with requirements, adjust priorities, and integrate

any open-source code references. By day 30, have a running core app with user login, and basic student-attendance-fee flows, ready for the next phase of development.

Mermaid Diagram – Roadmap: The Gantt chart above shows the high-level timeline for our roadmap.

Sources: Key claims were drawn from official and reputable sources: UDISE+ (MHRD/DoSEL) explains the data system for schools [3†L38-L43] [92†L42-L44] ; CBSE Affiliation Bye-laws detail required records [9†L270-L278] ; experts note that features like grades and attendance predict student outcomes [31†L227-L231] . Open-source project details (language, license, commit activity) come from GitHub and official sites [34†L207-L212] [55†L212-L220] [58†L87-L90] [66†L66-L69] [68†L444-L447] . Architecturally, industry sources advise starting with a shared-schema monolith for multi-tenancy [25†L64-L69] and building a plugin architecture for extensibility [28†L178-L188] . These insights underpin our recommendations.
